

Fingerprint Morphology

John Wensink

CSC515 Foundations of Computer Vision

Colorado State University – Global Campus

Dr. Bingdong Li

April, 20, 2025

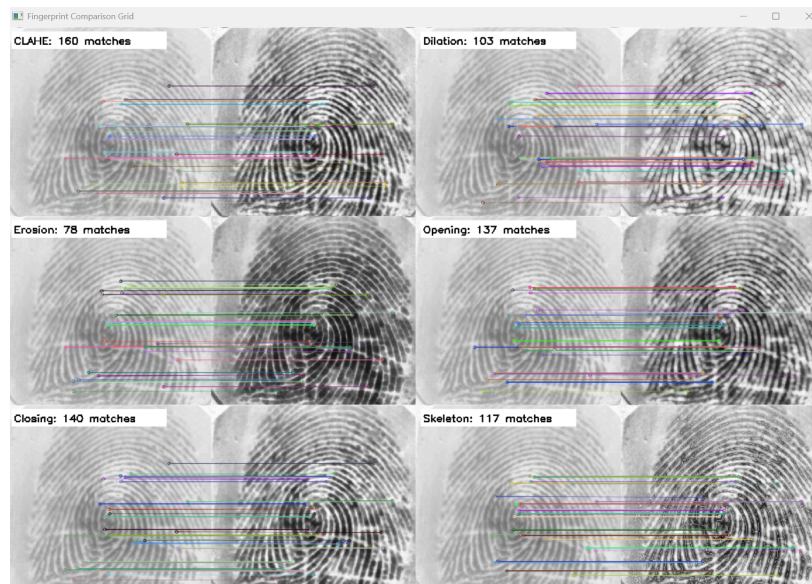
Fingerprint Morphology

Fingerprint recognition offers a compelling real-world use case for morphological transformations using OpenCV. In this project, I developed a Python-based pipeline to process a latent fingerprint image using a series of enhancement techniques and morphological operations, with the goal of improving ridge clarity, feature consistency, and reducing noise in an image selected for its low quality, significant smudging, and fragmented ridge structure. The process began by loading the fingerprint into a single-channel, greyscale image and applying Contrast-Limited Adaptive Histogram Equalization (CLAHE) preprocessing. This step improves local contrast and detail visibility by enhancing differences between the fingerprint ridges and valleys, especially in unevenly illuminated or smudged areas (GeeksforGeeks, 2023.) The use of CLAHE was useful for revealing ridge structures that might have been too faint for detection otherwise. The enhanced fingerprint was then subjected to five morphological transformations: dilation, erosion, opening, closing, and skeletonization. These operations were each applied using an elliptical structuring element with a 5x5 kernel, as elliptical shapes seem better suited for the naturally curved structure of fingerprint ridges (Shimat, n.d.) Dilation was applied with OpenCV's *cv2.dilate()*, which expanded the foreground ridge areas, effectively bridging minor gaps and improving continuity in broken ridge lines. This carries the side-effect of thicker ridge lines, which could potentially mask finer details (Morgan, 2023-b.) Erosion, in contrast, reduced the thickness of the ridges and eliminated small noise artifacts from the foreground by shrinking the white pixel regions. This was implemented using *cv2.erode()*, and while useful for clearing background clutter, it risked breaking weaker ridge connections or eliminating short ridge endings (Morgan, 2023-a.) To balance the strengths of dilation and erosion, opening and closing morphologies were applied using *cv2.morphologyEx()*. Opening uses an erosion transformation

followed by a dilation transformation to remove isolated noise elements while preserving the main fingerprint structure (TutorialsPoint, 2021.) Closing, on the other hand, involves a dilation operation followed by erosion, to fill in small holes or valleys in ridge structures that commonly occur in fingerprinting due to dryness or incomplete deposition of latent prints (Schadev, 2024.) Skeletonization was the final morphological transformation and was applied after converting the CLAHE-enhanced image into a binary format using Otsu's thresholding method (Helmy, 2023.) This approach automatically computes an optimal threshold value for binarization and ensures that ridges are represented as foreground pixels. The binary image was then inverted to match the input requirements of *skeletonize()* from *skimage.morphology*, which expects a white foreground and a black background. The result was a one-pixel-wide representation of the fingerprint ridge paths, preserving the overall topology of the features while minimizing pixel redundancy. To aid in visualization, the skeleton was colorized in orange and overlaid onto the CLAHE-enhanced image using OpenCV's *addWeighted()* function. I used an LLM (Grimoire, 2025) to help me get the skeletonized image to match the style of the other techniques.



Each transformation was compared to the original using ORB keypoint detection and feature matching (GeeksforGeeks, 2024.) Keypoints and descriptors were extracted using `cv2.ORB_create()`, and matches were determined using a brute-force matcher with the Hamming distance metric (ScienceDirect, n.d.) Matches were filtered using Lowe's ratio test method (Shah, n.d.), and refined further with homography estimation via RANSAC (Massachusetts Institute of Technology, 2021.) The number of valid matches was displayed on each comparison image, offering a quantitative evaluation of how well each transformation preserved the original fingerprint's features. In cases where transformations introduced distortion or lost structures, the match count decreased accordingly. The final visualization consisted of a grid layout generated with NumPy, displaying each match comparison side by side. This included CLAHE by itself, as well as each of the five morphological transformations explored in this project. Each image was labeled with its transformation name and the number of ORB matches detected between it and the original. The number of match lines was limited to 40 to avoid visual clutter that was obscuring the images. This layout provided a clearer idea of which enhancements preserved feature integrity and which may have degraded matchability.



Overall, the project successfully demonstrated how classical morphological operations paired with modern feature-matching algorithms can be used to assess and enhance fingerprint images. CLAHE by itself was the most effective in improving contrast and preserving details across the board. Dilation and closing helped reinforce ridge continuity, while erosion and opening were more effective at noise suppression. Skeletonization provided a minimalistic yet topologically accurate rendering of the ridge structures, which was visually appealing yet yielded fewer ORB matches. These findings aligned with the established literature on fingerprint enhancement and showed how the interplay of enhancement and analysis techniques can inform better biometric preprocessing pipelines.

References

GeeksforGeeks. (2023, May 19). CLAHE Histogram Equalization – OpenCV. GeeksforGeeks.

Retrieved April 20, 2025, from

<https://www.geeksforgeeks.org/clahe-histogram-equalization-opencv/>

GeeksforGeeks. (2024, June 4). Feature Matching Using ORB Algorithm in Python - OpenCV.

GeeksforGeeks. Retrieved April 20, 2025, from

<https://www.geeksforgeeks.org/feature-matching-using-orb-algorithm-in-python-opencv/>

Grimoire. (2025, April 20). Chat about fingerprint skeletonization, OpenCV, and morphological

operations [AI conversation]. ChatGPT. Retrieved April 20, 2025, from

<https://chat.openai.com/>

Helmy, B. E.-D. (2023, May 6). Understanding Otsu's Method for Image Segmentation.

Baeldung. Retrieved April 20, 2025, from

<https://www.baeldung.com/cs/otsu-segmentation>

Massachusetts Institute of Technology. (2012). Lecture 13: RANSAC. MIT 6.869 Advances in

Computer Vision. Retrieved April 20, 2025, from

<http://6.869.csail.mit.edu/fa12/lectures/lecture13ransac/lecture13ransac.pdf>

Morgan, J. (2023, March 30). Applying erosion to images with OpenCV: A complete guide.

Jeremy Morgan. Retrieved April 20, 2025, from

<https://www.jeremymorgan.com/tutorials/opencv/erosion-opencv-python/>

Morgan, J. (2023, March 31). Dilation Demystified: A Complete Guide to Image Dilation with

OpenCV. Jeremy Morgan. Retrieved April 20, 2025, from

<https://www.jeremymorgan.com/tutorials/opencv/dilate-opencv-python/>

Sachdev, A. (2024, March 17). Closing (Morphological Operation) — Image Processing.

Medium. Retrieved April 20, 2025, from

<https://medium.com/@anshul16/closing-morphological-operation-image-processing-59a0ef6210e3>

ScienceDirect. (n.d.). Hamming distance. Retrieved April 20, 2025, from

<https://www.sciencedirect.com/topics/computer-science/hamming-distance>

Shah, S. (2016). Project 2: Feature Matching. Georgia Institute of Technology. Retrieved April

20, 2025, from

https://sites.cc.gatech.edu/classes/AY2016/cs4476_fall/results/proj2/html/ssshah426/index.html

Shimat. (n.d.). Cv2.GetStructuringElement Method (MorphShapes, Size). OpenCvSharp

Documented Class Library. Retrieved April 20, 2025, from

https://shimat.github.io/opencvsharp_docs/html/a83a9f62-e3dc-c3e9-36bd-1758462d2198.htm

TutorialsPoint. (2021, March 17). Performing an opening operation on an image using OpenCV.

Retrieved April 20, 2025, from

<https://www.tutorialspoint.com/performing-an-opening-operation-on-an-image-using-opencv>